pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

The Daily Market Snapshot

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

One page, one date

Where the curve and spreads closed, how they moved, and where each sits in its own recent history: one page for Friday, October 2, 2026.

Then the same code, unchanged, for September 30: one command, any date.

No key today. Built from the frozen files, so it reruns to the same numbers anywhere.

Week 3: SQL, AI, and the snapshot

Day 11	SQL with sqlite3
Day 12	SQL joins, time series
Day 13	An AI assistant
Day 14	**The daily snapshot**
Day 15	The capstone

What goes on a one-page snapshot

Section	What it shows	Describes	Never used
`levels`	2, 5, 10, 30 Yr; two synthetic spreads	was, rose, fell	will, expect
`curve`	every tenor, today and the day before	up 4 bp on the day	due to revert
`slopes_flies`	2s10s, 5s30s, 3m10y, two flies	z-score 3.1, 252 rows	stretched, signal
book	DV01 and a what-if, synthetic book	99.6% at or below	cheap, rich, buy

The rule: a snapshot describes. What each number was, how it changed, where it sits in its own history. Never what it means or what to do.

snapshot_tables: four tables, one date

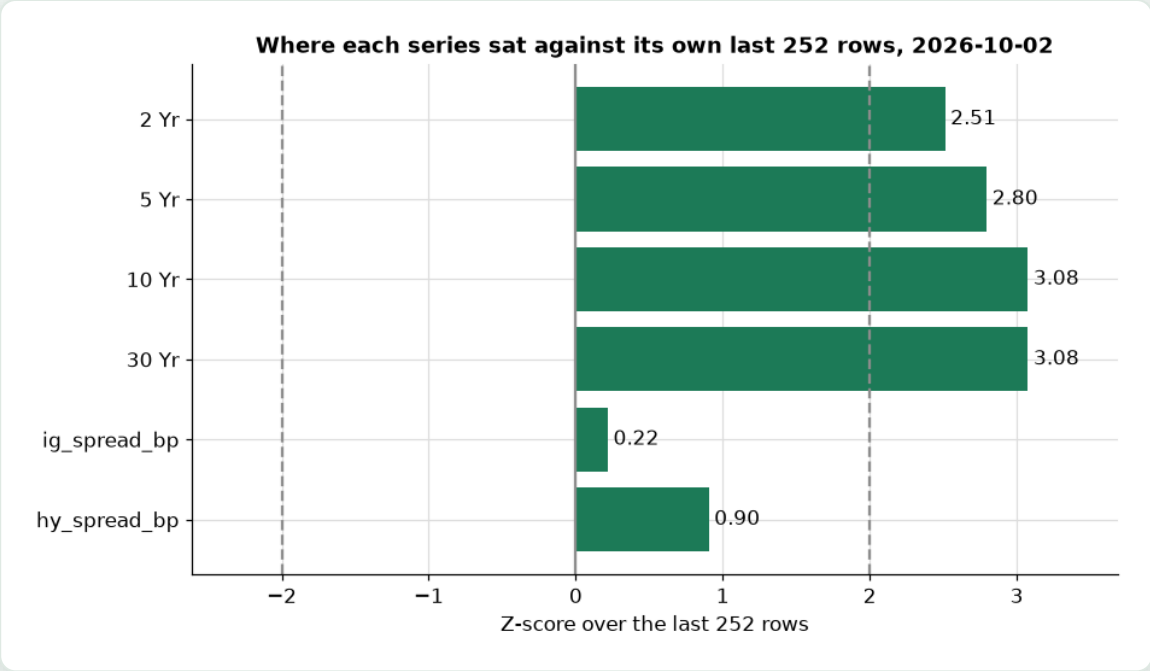
```
tables = md.snapshot_tables(
    market, book, "2026-10-02", SNAPSHOT_UNITS)
# inside: only what was known that day
known = market.loc[:as_of]
# and the date, for the writers
table.attrs["as_of"] = "2026-10-02"
```

series	level	1d bp	z	pct rank
2 Yr	4.83	+5	2.51	98.0
5 Yr	5.06	+5	2.80	99.6
10 Yr	5.28	+4	3.08	99.6
30 Yr	5.63	+2	3.08	99.6
ig_spread_bp	97.0	-1.0	0.22	61.1
hy_spread_bp	328.7	-1.3	0.90	73.8

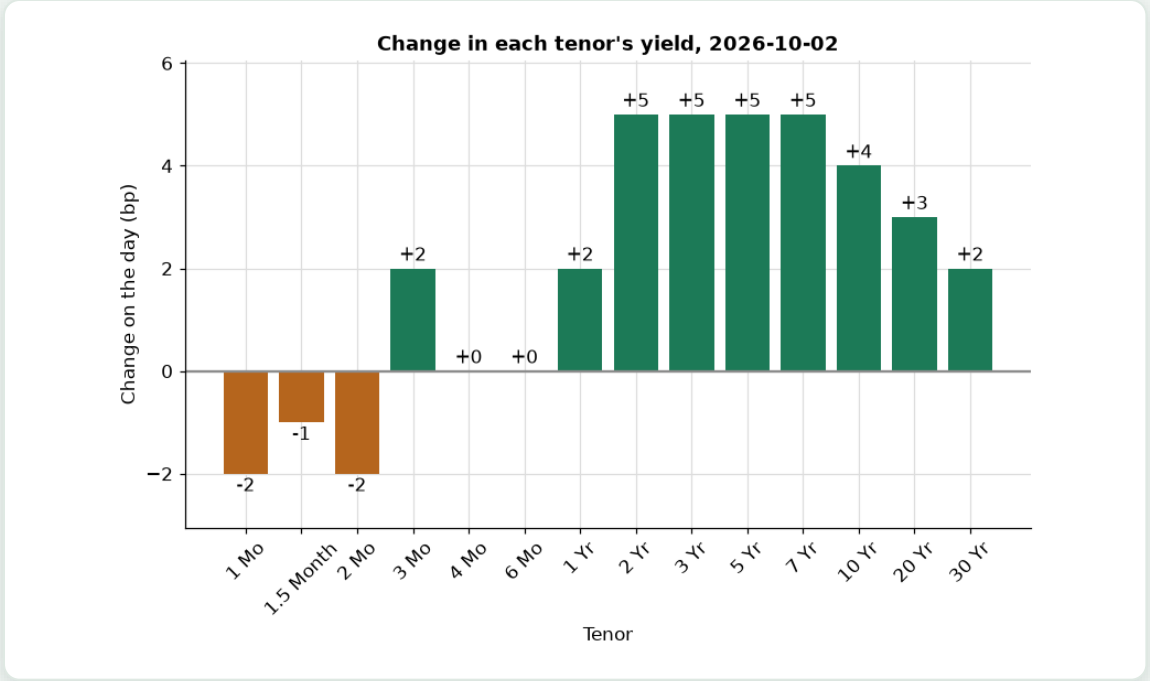
Built from the whole file, or from the file cut at its own date: the September 30 snapshot is identical, table by table.

2026-09-07 (Labor Day, no row): ValueError, not a borrowed day.

The levels and the curve, drawn

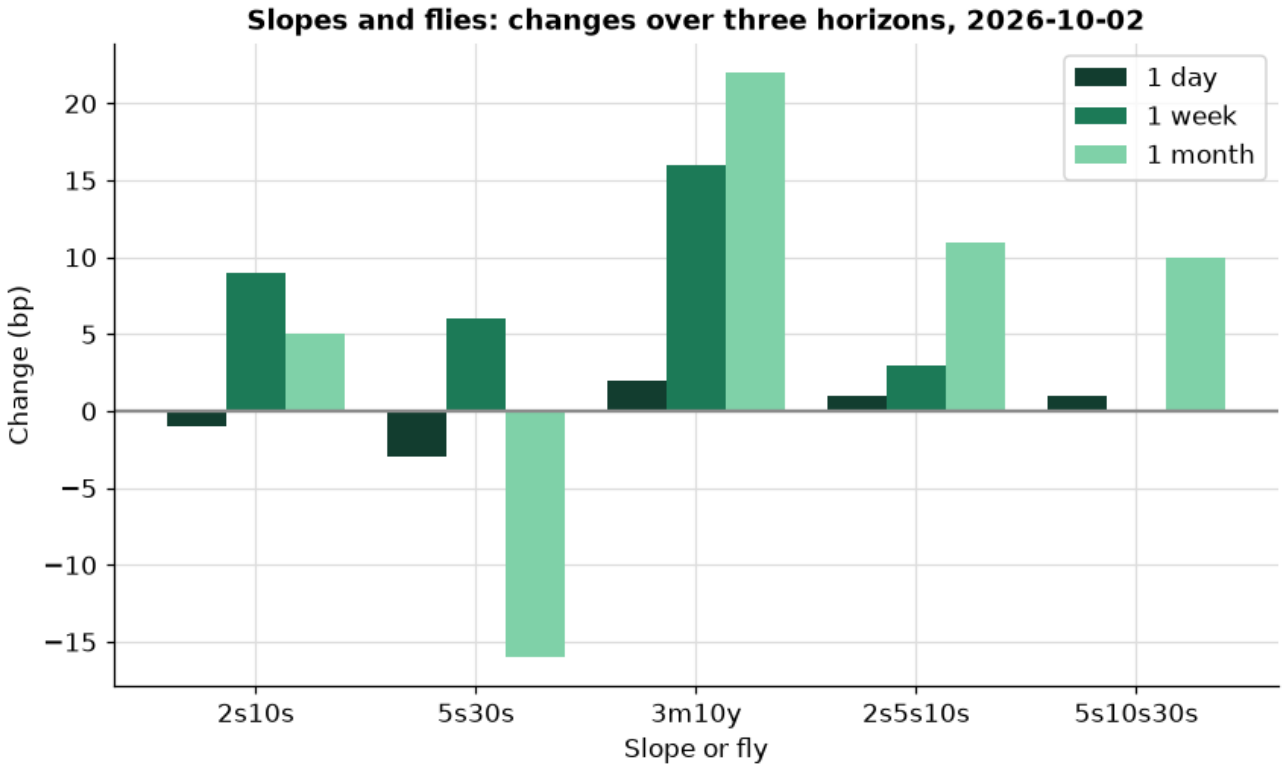


A bar is how many standard deviations the level sat from its 252-row average; the 10 Yr's was 3.08. It describes where the level was in its own year.



The shape of the day: where yields fell, where they rose, and by how many bp.

Slopes and flies, drawn



Three bars for each shape: its change over a day, a week, and a month, in bp.

This is description. It says what happened, not what it means.

The book: DV01 and a what-if

306,996.11

dollars per bp: the book's DV01, at the book's own date

-1,193,297.89

dollars: the mechanical change for the day's curve move

3.89 bp

change / DV01: the DV01-weighted average move

1. DV01

`bondmath.dv01 x face / 100, per bond`

2. The day's move

`chg_bp interpolated at each maturity`

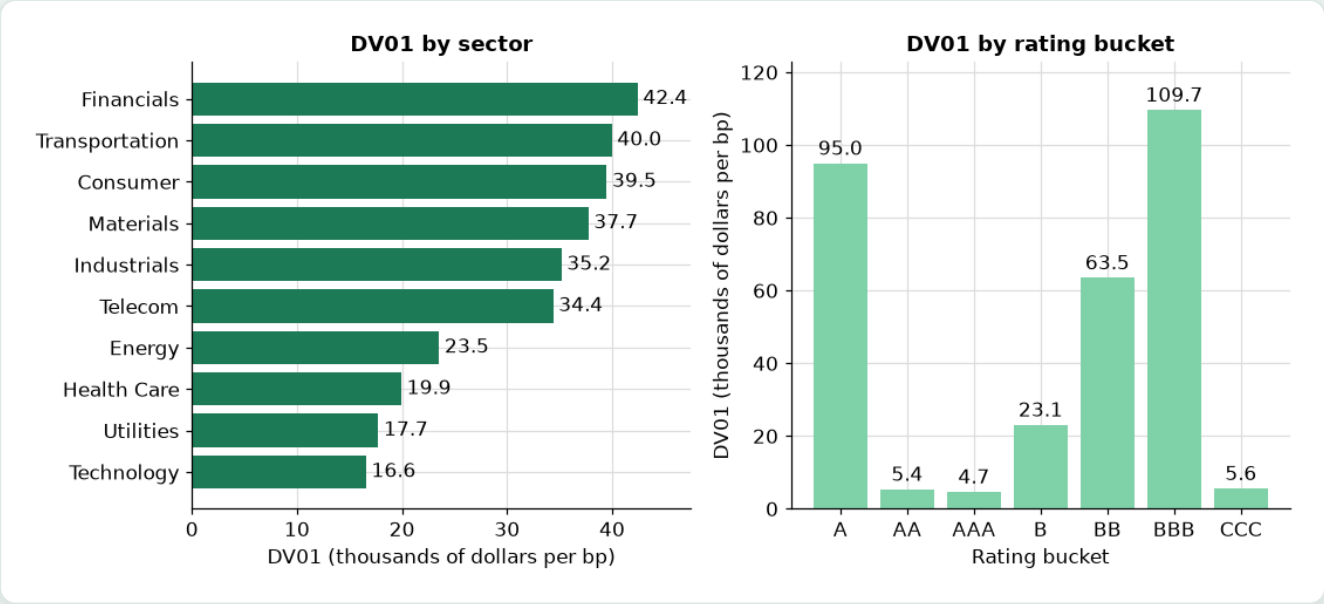
3. Reprice

`dirty_price at yield + move, x face / 100`

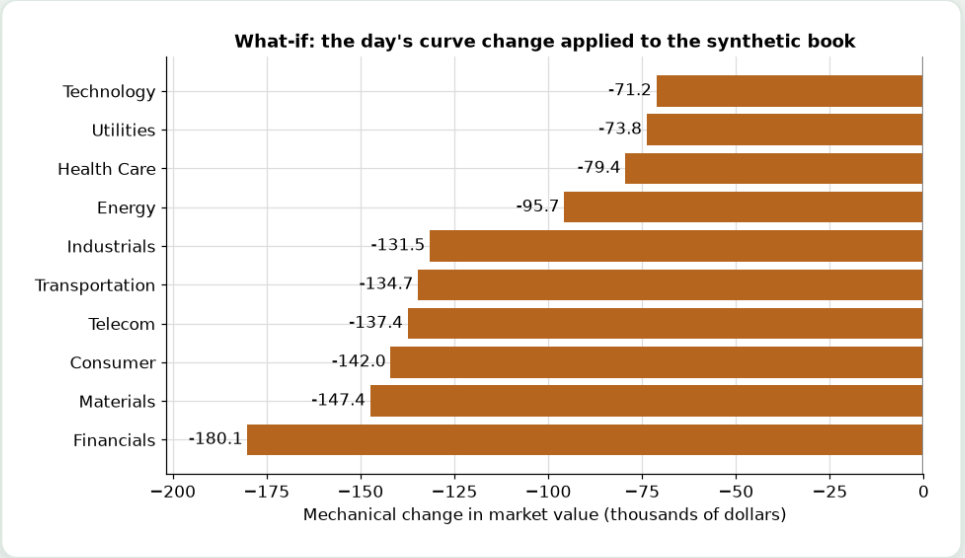
A what-if, not a profit and loss: one real curve move on a synthetic book. No spread change, no carry, no trading.

200 bonds, 489,000,000 dollars of face; sectors and buckets each add back to the book, to the cent.

The book, drawn

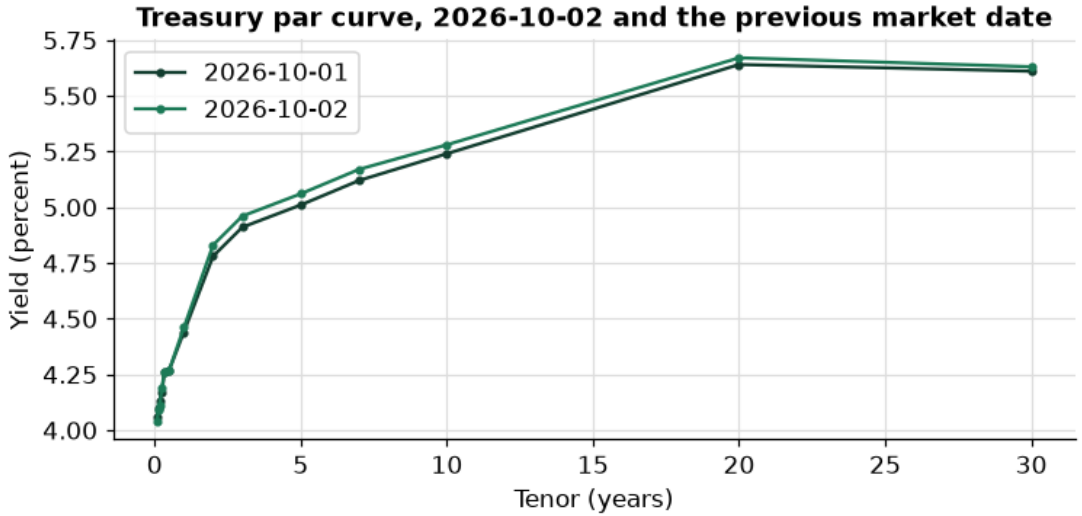


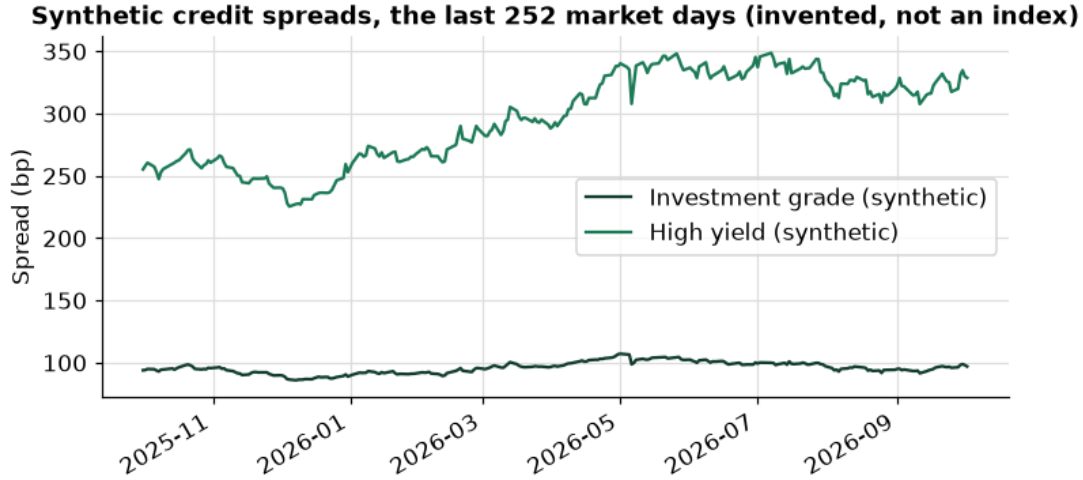
Two splits of the same total DV01: by sector, and by rating bucket.



A what-if, not a profit and loss: only the Treasury curve change moves.

Two charts, both labelled





The spread chart is the same 252 rows as the z-score and the percentile rank, and every label says synthetic.

One page, one workbook, both read back

write_snapshot_html: one file

Tables by to_html, charts as base64 PNGs inside the page, no template library.

```
headings read back:
levels, curve, slopes_flies, book
Treasury curve, Synthetic credit spreads
tables 4, charts 2, path or user name: none
```

write_snapshot_excel: about, then tables

Opened in Excel: about, levels, curve, slopes_flies, book. General format, no ####.

Book face shows 4.89E+08. Consumer's market value shows 56629598; the cell holds every digit.

The about sheet carries the as-of date and the synthetic label.

Read back with html.parser and pd.read_excel, then four new tests: 55 passed.

snapshot.py: one command, any date

`argparse`

--as-of from the command line, and --help for free

`HERE`

finds its data from its own folder

`Figure`

charts drawn with no screen

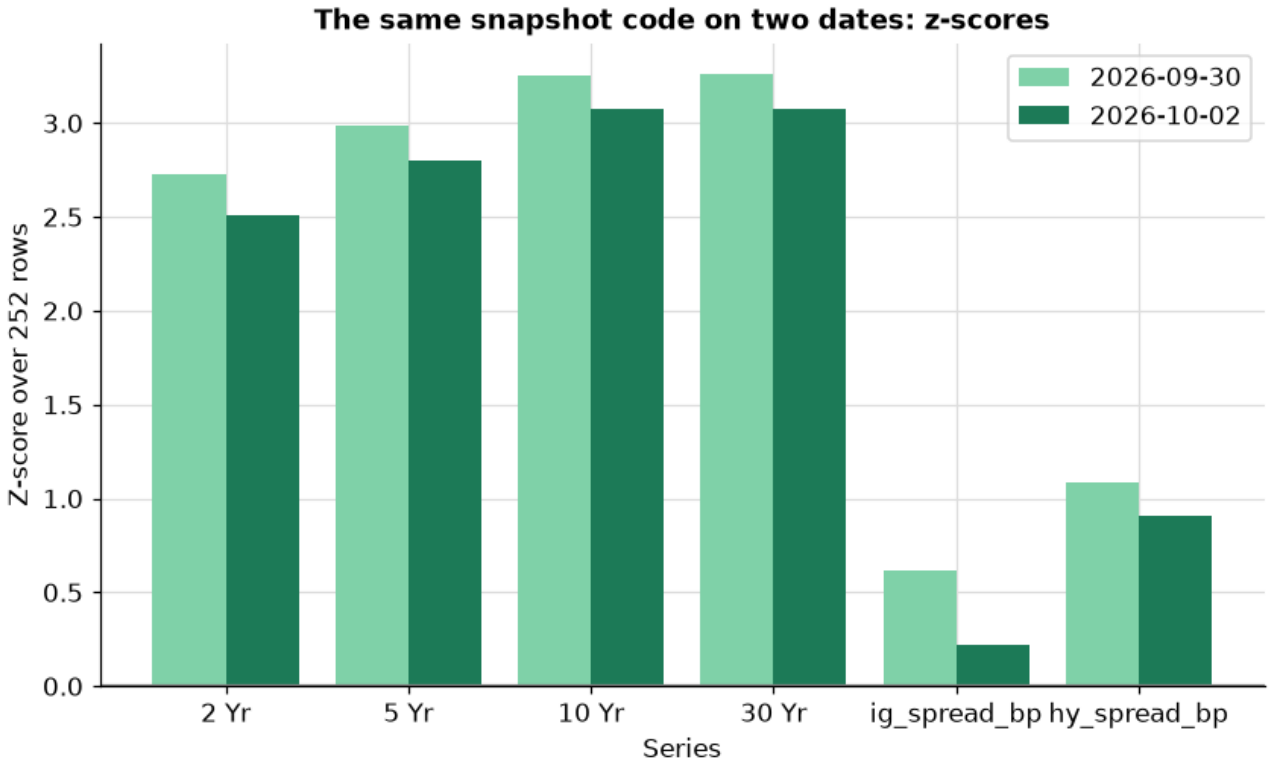
`parser.error`

a bad date: one line, exit code 2

```
python snapshot.py --as-of 2026-10-02
snapshot for 2026-10-02: wrote snapshots/snapshot_2026-10-02.html and snapshots/snapshot_2026-10-02.xlsx
python snapshot.py --as-of 2026-09-07
exit code 2:  snapshot.py: error: 2026-09-07 is not a market date: market has no row for it
```

Rerun, do not redo: two dates side by side, and not one line of code changed.

Two dates, one function



The same series in the two reports, read back from the two workbooks. Not one line of code changed between them.

Rerun, do not redo.

Every day: Windows Task Scheduler

```
# with the course's .venv active (shown, not run)
$python = (Get-Command python).Source
schtasks /Create /TN "pfi_daily_snapshot" /SC DAILY /ST 18:30 /TR "'$python' '...\snapshot.py'"
schtasks /Query /TN "pfi_daily_snapshot"
schtasks /Run /TN "pfi_daily_snapshot"
schtasks /Delete /TN "pfi_daily_snapshot" /F
```

Every option is from schtasks' own help: /TN name, /SC schedule, /ST start time (HH:mm), /TR command, inner quotes single.

At work, your firm's policy decides whether scheduled tasks may run on your machine. Follow it.

Excel side by side

Excel side by side (tested in Excel)

`=(D2-C2)*100`	on curve: matches chg_bp within 8.9e-16 bp
`=SUMIFS(G2:G19,B2:B19,"sector")`	306,996.11: the book row's DV01
`=SUMIFS(G2:G19,B2:B19,"rating bucket")`	306,996.11
`=STANDARDIZE(A252,AVERAGE(A1:A252),STDEV.S(A1:A252))`	3.075234
`=COUNTIF(A1:A252,"<="&A252)/COUNT(A1:A252)*100`	99.60317

On the snapshot workbook itself: the curve and book sheets as written, and the 10 Yr's last 252 yields in A1:A252 of a new sheet.

Tested in Excel 16.0 build 20430, 2026-10-04: opened with Workbooks.Open, closed without saving.

Save the day: the script, not the outputs

1. Commit the day

marketdata.py, the tests, .gitignore

2. Ignore outputs

snapshots/ added to .gitignore: !!

3. Commit the script

snapshot.py and the .gitignore line

```
git log --stat -1
.gitignore  |  2 ++
snapshot.py | 84 +++...
```

The outputs are rebuilt by one command, so they never enter the record.

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: snapshot_levels takes market.iloc[-1] for today: asked for September 30, it shows October 2.

Code that runs is not the same as code that is right.

What you can do now

- List what goes on a one-page snapshot, and keep it descriptive
- Build every table for one date from rows on or before it only
- Read a book's DV01 and a mechanical what-if, labelled as one
- Write a page and a workbook from the same tables, and check both
- Run snapshot.py for any date; schedule it; commit code, not outputs

Week 3, Day 5 The capstone: a vendor extract cleaned, proved, and run through today's snapshot